Parallelized K-Means Clustering for Efficient Data Analysis

Presented by-

Vaibhav Barsaiyan [cs24mtech12008]
Subir Kumar Paul [cs24resch11012]

Guided by-

Dr. Sathya Peri



భారతీయ సాంకేతిక విజ్ఞాన సంస్థ హైదరాబాద్
भारतीय प्रौद्योगिकी संस्थान हैदराबाद
Indian Institute of Technology Hyderabad

INDEX



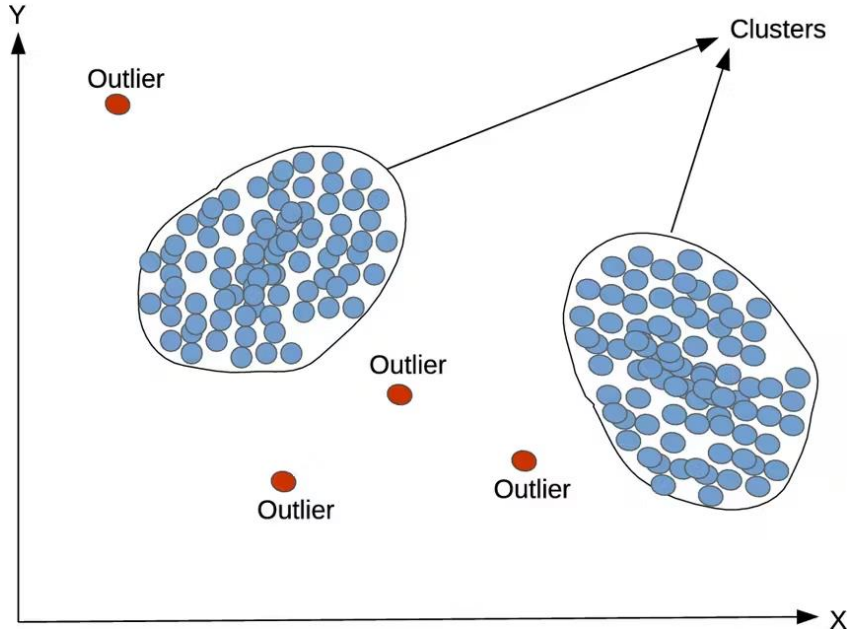
- Operations on data
- Introduction to K-means & its Impact
- Why K-means algo for clustering ?
- Combination with Outlier detection algo and its Benefits
- Mathematical background and Formulae used
- Steps used for Parallelization
- Explaining of Overall Aspects of Project
- Literature Review



OPERATIONS ON DATA

Data Preprocessing : It involves preparing raw data for analysis by cleaning, transforming, and organizing it to improve the quality and relevance of the information.

Data Clustering : Clustering is an unsupervised machine learning technique used to group similar data points into clusters based on their characteristics.



K-Means Clustering algorithm



Purpose : Specifically designed for clustering the tasks.

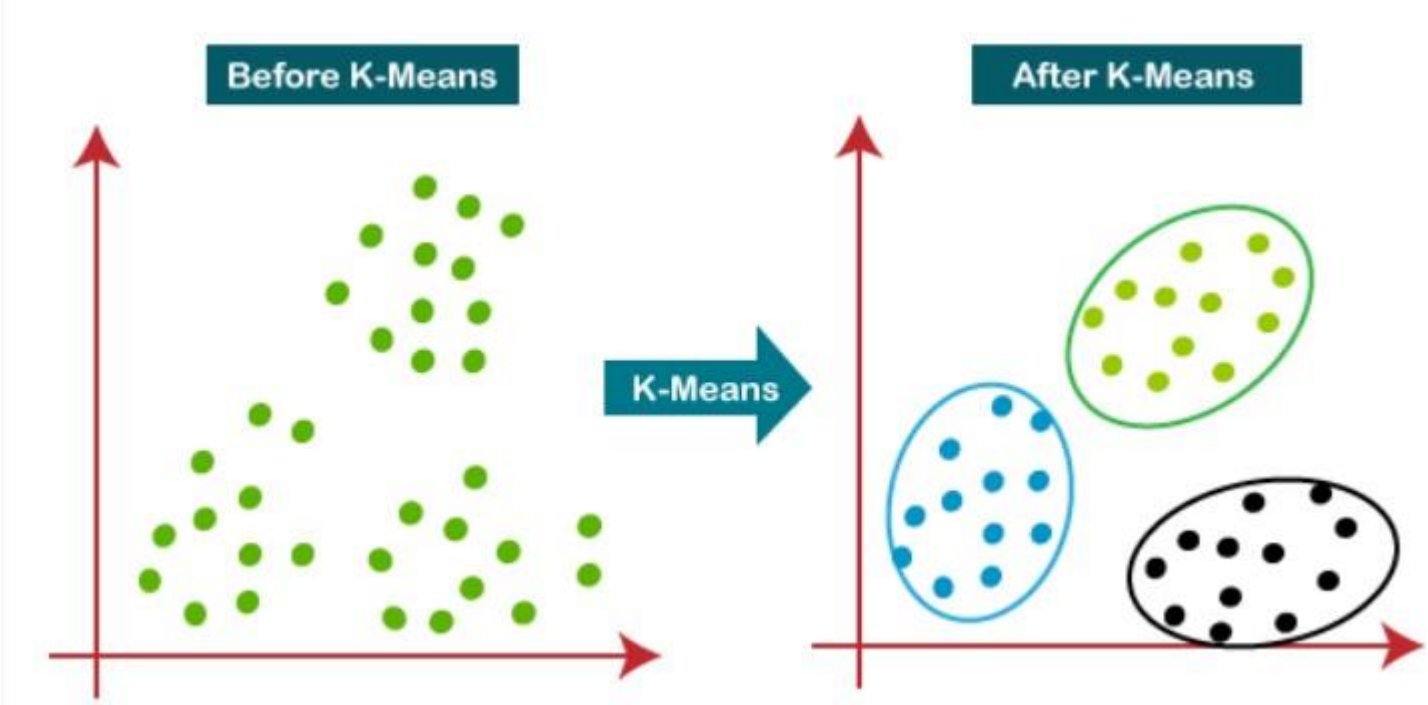
Working : K-Means groups data points into clusters by minimizing the distance between points and their respective cluster centers (centroids). It iteratively refines these clusters until the optimal division is found.

Best for : When we need to discover groups (clusters) in our data without any labels.

Example : Grouping customers into segments based on their purchasing behavior.



Impact of K-means



Why selecting K-means clustering ?



1] Since outlier identification algorithms like the isolation forest algorithm may work well with both supervised and unsupervised datasets, the data clustering algorithm works well only with the unsupervised dataset.

2] As our task involves using an unsupervised dataset , where K-means clustering is better and more efficient to work with compared to KNN algorithms.



Combination of K-Means Clustering & Isolation Forest

K-Means for Clustering:

- **Purpose:** Helps group similar data points into clusters based on their features.
- **Benefit:** After clustering, we can analyze each cluster separately, which may help us discover distinct patterns and detect anomalies within each cluster.

Isolation Forest for Outlier Detection:

- **Purpose:** Detects data points that are outliers or anomalies, even when clusters exist.
- **Benefit:** Once the data is clustered, we can use Isolation Forest to identify outliers either within individual clusters or across the entire dataset.



Benefits of this Combination:



1] Improved accuracy in detecting combining anomalies :
ensures data is well organized into meaningful clusters, making it easier for Isolation Forest to spot outliers.

2] Handling mixed distributions :
Clustering first allows you to handle cases where the data distribution may vary across different clusters, making outlier detection more precise.

This combination provides a more structured and comprehensive approach for the handling of large datasets with complex patterns.



Mathematical Concepts in Parallel K-Means Clustering

1. **Cluster Assignment Formula:**

   $$C_i = \arg\min_k \|x_i - \mu_k\|^2$$

2. **Centroid Update Formula:**

   $$\mu_k = \frac{1}{|C_k|} \sum_{x_i \in C_k} x_i$$

3. **Euclidean Distance Formula:**

   $$d(x_i, \mu_k)^2 = \sum_{j=1}^{m} (x_{ij} - \mu_{kj})^2$$

4. **Convergence Check Formula:**

   $$\|\mu_k^{(t+1)} - \mu_k^{(t)}\| < \epsilon$$



Abbreviations



1. C_i - Cluster assigned to data point x_i.
2. x_i - i-th data point in the dataset.
3. μ_k - Centroid of cluster k.
4. $d(x_i, \mu_k)$ - Euclidean distance between data point x_i and centroid μ_k.
5. $|C_k|$ - Number of data points in cluster k.
6. x_{ij} - j-th feature of data point x_i.
7. μ_{kj} - j-th feature of centroid μ_k.
8. m - Total number of features in each data point.
9. ϵ - Small threshold value for convergence check.
10. t - Iteration number during the clustering process.



Steps for Parallelization:



- **Step 1:** Split the dataset into chunks. Each chunk is assigned to a separate thread or process.
- **Step 2:** In parallel, each thread computes the distance between the points in its chunk and all centroids.
- **Step 3:** Each thread assigns its points to the nearest centroid and collects results.
- **Step 4:** After all threads finish, combine the results to form new centroids for next iteration.
- **Step 5:** After Recalculating the centroids, again apply the K-means algorithm among different threads separately parallely.



Pros & cons - Parallelizing Distance Calculation in K-Means

PROS -

- Significant Speedup
- Scalability with Larger Datasets
- Better Resource Utilization

CONS -

- Increased Complexity
- Load Balancing Issues
- Potential Overhead





NOTE →

In some of the next slides, I try explaining basic points implemented in our project of parallelized K-means clustering algorithm for efficient data analysis.

[HOPE this section will clarify most of the important points regarding this project work]



Part - 1

1. How is the optimal number of clusters selected in the code?

- **Answer**: The number of clusters (numClusters) is hardcoded to 5 at the beginning of the program. The choice of 5 could be based on prior knowledge or experimentation.
- **Code Snippet**:

  int numClusters = 5;

- **Example**: The code will always try to create 5 clusters for any given dataset.

---

2. Why did the I selected the number of clusters as 5?

- **Answer**: The number of clusters (5) is predefined. It could be adjusted based on domain knowledge or through methods like the Elbow Method or Silhouette Score.

- **Code Snippet**:

  int numClusters = 5; // Set at the beginning

Part - 2



3. Can we change the number of clusters and execute the code without any issues?

- **Answer**: Yes, the number of clusters can be changed in the code, but it should be consistent throughout the program to avoid mismatches between data structures.

- **Code Snippet**:

  ```
  vector<Centroid> centroids(numClusters);
  ```

---

4. What is the best way to determine the number of clusters dynamically or hardcoded?

- **Answer**: Dynamically, you can use methods like the Elbow Method or Silhouette Score to determine the optimal number of clusters based on the dataset.



Part - 3

5. How does the parallel K-means algorithm solving the convergence issue?

- **Answer**: The algorithm iterates over points, assigns them to clusters, and updates centroids until convergence is reached. It avoids infinite execution by limiting iterations (maxIterations).
- **Code Snippet**:

```
for (int iter = 0; iter < maxIterations; iter++)
{
    // Assignment and centroid update logic
}
```

---

6. Can a centroid be left unused or have no datapoints?

- **Answer**: Yes, if a centroid does not attract any points, it remains unchanged. This is handled by recalculating centroids based on the points assigned to them.
- **Code Snippet**:

```
centroids[c].x = sumX / count;  // If count is 0, no update happens
```

Part - 4



7. How are data points distributed to clusters at the beginning?

- **Answer**: Points are assigned to clusters based on the Euclidean distance to the centroids. Each point will belong to the nearest centroid.
- **Code Snippet**:

```
for (int c = 0; c < data->numClusters; c++) {
    double dist = pow(points[i].x - centroids[c].x, 2) + pow(points[i].y - centroids[c].y, 2);
}
```

---

8. Can the same datapoint be assigned to multiple clusters?

- **Answer**: No, a point can only belong to one cluster at a time. The nearest centroid is chosen based on the smallest distance.
- **Code Snippet**:

```
points[i].cluster = c;  // A point gets assigned to a single cluster
```



Part - 5



9. Summary of Data Structures used for various purpose in the project →

1. **Point Structure**: Stores coordinates in n-dimensional space using a vector<double>.
2. **Centroid Structure**: Stores coordinates in n-dimensional space using a vector<double>.
3. **ThreadData Structure**: Holds thread-specific data such as the points, centroids, and localSumX/Y vectors for centroid updates.
4. **vector<double>**: Used extensively for representing coordinates, sums of coordinates, and distances for arbitrary dimensions.
5. **vector<Point> and vector<Centroid>**: Store all the data points and centroids, respectively, where each point or centroid is represented as a vector of coordinates.
6. **vector<int>**: Used to store counts of points in each cluster for centroid update.



Part - 6

10. How are boundaries between clusters maintained?

- **Answer**: Boundaries are implicitly defined by the distance from the data points to the centroids. Each point belongs to the cluster with the nearest centroid.
- **Example**: For points near two centroids, the decision boundary will be at the midpoint.

---

11. How is it ensured that the same datapoint will only be assigned to one thread?

- **Answer**: Data points are chunked into smaller sections. Each thread processes its own chunk, ensuring no overlap of datapoints between threads.
- **Code Snippet**:

  threadData[t] = {&points, &centroids, {}, {}, {}, t * chunkSize, (t + 1) * chunkSize, numClusters};

---

12. How is chunking done in the code?

- **Answer**: The data is divided into chunks where each thread works on a specific range of indices. Each thread processes its own set of points independently.
- **Code Snippet**:

  int chunkSize = n / numThreads;

Part - 7

13. How are data points stored and chunked?

- **Answer**: Data points are stored in a vector<Point>. The chunking is done by dividing the dataset into equal parts and assigning each part to a different thread.
- **Code Snippet**:

  vector<Point> points(dataSetSize);

14. On what basis is a data point assigned to a thread?

- **Answer**: Data points are divided by index range (not based on X or Y values). Each thread processes a specific range of indices in the dataset.
- **Code Snippet**:

  threadData[t] = {&points, &centroids, {}, {}, {}, t * chunkSize, (t + 1) * chunkSize, numClusters};

Part - 8

15. Explain the chunking process with an example?

- **Answer**: If there are 100 data points and 4 threads, each thread will process 25 points. Points 0-24 will be processed by thread 1, and so on.
- **Code Snippet**:

```
for (int t = 0; t < numThreads; t++) {
    threadData[t] = {&points, &centroids, {}, {}, {}, t * chunkSize, (t + 1) * chunkSize, numClusters};
}
```

---

16. What is the role of the vector point data structure?

- **Answer**: The vector<Point> stores the X and Y coordinates of each point. The vector<Centroid> stores the X and Y coordinates of each centroid.
- **Code Snippet**:

```
struct Point {
    double x, y;
    int cluster;
};
```

Part - 9

17. How is non-assignment of a data point to a cluster avoided?

- **Answer**: Points are always assigned to the nearest centroid during each iteration, ensuring each point is assigned to a valid cluster.
- **Code Snippet**:

```
points[i].cluster = c;  // Ensure every point has a cluster
```

---

18. Can the code handle n-dimensional points?

- **Answer**: Yes, by modifying the Point structure to store multiple dimensions and adjusting the distance calculation accordingly.
- **Code Snippet**:

```
struct Point {
    vector<double> coordinates;  // Store n-dimensional points
};
```

Part - 10

19. What data structure is used for n-dimensional points?

- **Answer**: A vector<double> can be used to store n-dimensional coordinates.
- **Code Snippet**:

```
struct Point {
    vector<double> coordinates;  // For n-dimensional points
};
```

---

20. Where is the barrier to stop threads before the next iteration?

- **Answer**: Threads stop after completing their task (either assigning clusters or updating centroids) using pthread_join.
- **Code Snippet**:

```
pthread_join(threads[t], nullptr);
```

Part - 11

21. Can atomic locks replace mutex locks?

- **Answer**: Yes, atomic operations can replace mutex locks for simple operations but may not work well for complex updates like centroid calculations.
- **Code Snippet**: Atomic operations could be used in place of mutex if necessary.

---

22. How is the validation of datapoint assignments handled during convergence?

- **Answer**: The algorithm checks if the centroid positions have changed significantly. If not, it stops the iteration.
- **Code Snippet**: This can be done by checking the change in centroids after each iteration.

---

23. Can explicit validation techniques be used to validate cluster assignments?

- **Answer**: Yes, methods like the Silhouette Score or Davies-Bouldin Index can be used to validate the final clustering results.
- **Code Snippet**: You could implement the Silhouette Score to assess clustering quality.

Literature review

1] A Study of Clustering Techniques and Their Applications in Various Domains" covers practical applications of K-Means in fields like image processing, text analysis, and market segmentation .

2] A K-Means Clustering Algorithm" by J. B. MacQueen (1967)

3] An Efficient K-Means Clustering Algorithm: Analysis and Implementation" by Tapas Kanungo et al. (2002)

4] Efficient Implementation of KNN for Big Data Using Hadoop" by Maryani et al. (2014)

THANK YOU